

A HANDS-ON, CODE-FIRST COURSE

LangChain From Zero

Build real GenAI apps with Gemini & local models

PHASE 5 — MEMORY & CONVERSATIONAL APPS

Prerequisites: Phases 1–4 done; `MessagesPlaceholder` (Phase 3) fresh in mind; `langgraph` installed (Phase 0).

Targets: stateless models · message history · LangGraph checkpointing & `thread_id` · context management

Estimated time for this phase: 80–120 minutes

Where this phase sits

A model forgets everything between calls. This phase gives a conversation **memory** — first by hand so you understand the mechanism, then with the **modern LangGraph checkpoint** that does it properly (and replaces the deprecated memory classes you'll see in old tutorials).

Ph 0 ✓ **Environment Setup**

Ph 1 ✓ **LangChain Fundamentals**

Ph 2 ✓ **Working with Models**

Ph 3 ✓ **Prompts & Output Parsers**

Ph 4 ✓ **Chains & LCEL Composition** — parallel, routing, custom Python, debugging.

Ph 5 **Memory & Conversational Apps** ← you are here
Stateless models, message history, LangGraph persistence (checkpoint + thread_id), context management.

Ph 6–8 **Documents · Embeddings · RAG**

Ph 9–12 **Tools & Agents · LangGraph · Deployment · Capstone**

Phase 5 · Memory & Conversational Apps

WHAT YOU'LL BE ABLE TO DO AFTER THIS PHASE

Explain why models are stateless and what "memory" really is; build a remembering chatbot by hand with a message list; then build one the modern way with a LangGraph **checkpoint** and **thread_id** — including separating conversations per user — on both Gemini and Ollama. You'll also know which old memory APIs to avoid and why.

In this phase

1. 5.1 — Models are stateless: what "memory" actually means
2. 5.2 — The deprecated memory classes (and what replaced them)
3. 5.3 — Memory by hand: a growing message list
4. 5.4 — Why by-hand doesn't scale
5. 5.5 — Modern memory: the LangGraph checkpoint
6. 5.6 — Threads: keeping conversations separate
7. 5.7 — Keeping context from overflowing
8. 5.8 — Common errors & troubleshooting
9. 5.9 — Mini practice exercise
10. Terms introduced this phase

5.1 — Models are stateless: what "memory" actually means

Here's a surprise that trips up every beginner: **a model remembers nothing between calls**. Each `invoke` is a blank slate. If you ask "What's my name?" right after telling it, a fresh call has no idea — you never sent the earlier turn.

TERM: STATELESS

Stateless means the model keeps no record of past calls. The *only* thing it knows is exactly what you put in *this* request. "Memory," then, is not something inside the model — it's **you re-sending the conversation so far** with each new turn.

```
Turn 1:  send [ "Hi, I'm Sam" ]           → "Hello Sam!"
Turn 2:  send [ "What's my name?" ]       → "I don't know"   (no
history!)
Turn 2 fixed: send [ "Hi, I'm Sam", "Hello Sam!", "What's my name?" ] → "You're
Sam!"
```

So memory = accumulate the message list and resend it every turn. Everything in this phase is just *how* to manage that list well.

5.2 — The deprecated memory classes (and what replaced them)

LangChain's memory story changed significantly. Older tutorials (and a lot of blog posts) use classes that are now **deprecated**. Knowing the names saves you from copying dead code.

⚠ LEGACY — DO NOT USE THESE

Deprecated	Use instead (this phase)
<code>ConversationChain</code>	A LangGraph graph with a checkpointers
<code>ConversationBufferMemory</code> , <code>ConversationBufferWindowMemory</code> , <code>ConversationSummaryMemory</code>	LangGraph state + (optionally) trim / summarization middleware
<code>RunnableWithMessageHistory</code>	LangGraph checkpointers (thread-level persistence)

These still appear in search results because they were standard until recently. In LangChain 1.x, **memory lives in LangGraph's persistence layer**. That's what §5.5 teaches.

5.3 — Memory by hand: a growing message list

Before any framework magic, let's build memory ourselves so you see exactly what it is. Keep a Python list of messages; append each turn; resend the whole list every time.

```
from dotenv import load_dotenv
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.messages import HumanMessage

load_dotenv()
model = ChatGoogleGenerativeAI(model="gemini-2.5-flash")

history = [] # 1 our memory = a list of messages

def chat(user_text: str) -> str:
    history.append(HumanMessage(user_text)) # 2 record the user's turn
    reply = model.invoke(history) # 3 send the WHOLE history, not just this
    line
    history.append(reply) # 4 record the model's reply (an
    AIMessage)
    return reply.content

print(chat("Hi, my name is Sam."))
print(chat("What's my name?")) # works — history still holds turn 1
```

Line	Explanation
1	<code>history</code> is the memory: an ordinary list that will hold alternating Human/AI messages.

2	Before calling, append the new user message so it's part of what we send.
3	Pass the <i>entire</i> list to <code>invoke</code> . The model now "sees" every prior turn — that's the whole trick.
4	Append the model's reply too, so the next turn includes it. Memory grows by two messages per exchange.

Expected output:

```
Nice to meet you, Sam!
Your name is Sam.
```

THAT'S GENUINELY ALL MEMORY IS

No hidden state, no database — just a list you keep appending and resending. The Ollama version is the usual one-liner (`ChatOllama(model="llama3.2")` , drop `load_dotenv()`). If you combine this with the `MessagesPlaceholder` from Phase 3, you can also slot `history` into a templated prompt with a system persona — try it.

5.4 — Why by-hand doesn't scale

The manual approach is perfect for learning and fine for a quick script. But a real app hits walls fast:

- **No persistence.** The list lives in memory; restart the program and every conversation is gone.
- **No multiple users.** One global `history` means everyone shares one conversation. You'd need to juggle a dict of lists keyed by user, by hand.
- **No context control.** The list grows forever until it overflows the model's context window (and your bill).
- **You re-implement plumbing** that LangChain already solved — saving, loading, threading, trimming.

The modern primitive fixes all four with almost no extra code.

5.5 — Modern memory: the LangGraph checkpointer

TERMS: GRAPH, STATE, CHECKPOINTER, THREAD

- **Graph (LangGraph):** a small workflow of steps ("nodes"). For a chatbot we need just one node: "call the model." (Phase 10 builds richer graphs.)
- **State:** the data the graph carries — for chat, a `messages` list. LangGraph's prebuilt `MessagesState` gives you exactly that, and automatically *appends* new messages instead of replacing them.
- **Checkpoint:** the piece that saves the state after every step so it survives between calls. `InMemorySaver` for development; a database-backed one (SQLite/Postgres) for production.
- **Thread:** one independent conversation, identified by a `thread_id`. The checkpointer stores a separate history per thread.

Build a memory-backed chatbot. Notice you never manage the list yourself — the checkpointer does:

```

from dotenv import load_dotenv
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.messages import HumanMessage
from langgraph.graph import StateGraph, START, MessagesState      # 1
from langgraph.checkpoint.memory import InMemorySaver              # 2

load_dotenv()
model = ChatGoogleGenerativeAI(model="gemini-2.5-flash")

def call_model(state: MessagesState):                             # 3 the one node
    response = model.invoke(state["messages"])                    # send the whole history
    return {"messages": response}                                  # appended to state
                                                                    automatically

builder = StateGraph(MessagesState)                                # 4 build the graph
builder.add_node("model", call_model)
builder.add_edge(START, "model")                                   # start -> model node

chatbot = builder.compile(checkpointer=InMemorySaver())             # 5 attach memory

config = {"configurable": {"thread_id": "sam-1"}}                  # 6 which conversation

r1 = chatbot.invoke({"messages": [HumanMessage("Hi, I'm Sam.")]}, config)
print(r1["messages"][-1].content)

r2 = chatbot.invoke({"messages": [HumanMessage("What's my name?")]}, config) # 7
print(r2["messages"][-1].content)

```

Line	Explanation
1	<code>StateGraph</code> builds the workflow; <code>START</code> is the entry point; <code>MessagesState</code> is the ready-made state holding a <code>messages</code> list that appends.
2	<code>InMemorySaver</code> is the development checkpointer — it keeps state in RAM, keyed by thread.
3	The single node: take the current <code>messages</code> , call the model, return the reply. Returning <code>{"messages": response}</code> appends it (that's the magic of <code>MessagesState</code>), so you never manage the list.
4	Wire the graph: one node named <code>"model"</code> , with an edge from <code>START</code> into it.
5	<code>.compile(checkpointer=...)</code> turns the blueprint into a runnable chatbot <i>with memory attached</i> .
6	The <code>config</code> names the conversation via <code>thread_id</code> . Same id = same remembered history.
7	The second call sends only the new question — yet the checkpointer reloads turn 1, so the model still knows the name.

Expected output:

```

Hi Sam! Nice to meet you.
Your name is Sam.

```

WHAT YOU GOT FOR FREE

Persistence, automatic history appending, and per-thread separation — without a single line of list-juggling. Swap to Ollama with `ChatOllama(model="llama3.2")`; everything else is identical. For production, swap `InMemorySaver()` for a database checkpointer (e.g. `SqliteSaver` / `PostgresSaver`) and conversations survive restarts.

5.6 — Threads: keeping conversations separate

The `thread_id` is what makes this multi-user. Use a different id and you get a clean, separate memory — perfect for "one thread per user" or "one thread per chat session."

```
sam = {"configurable": {"thread_id": "sam-1"}}
alex = {"configurable": {"thread_id": "alex-2"}}      # a DIFFERENT conversation

chatbot.invoke({"messages": [HumanMessage("I'm Sam.")]}, sam)
chatbot.invoke({"messages": [HumanMessage("I'm Alex.")]}, alex)

print(chatbot.invoke({"messages": [HumanMessage("Who am I?")]}, sam )["messages"]
[-1].content)
print(chatbot.invoke({"messages": [HumanMessage("Who am I?")]}, alex)["messages"]
[-1].content)
```

Expected output — each thread remembers only its own turns:

```
You are Sam.
You are Alex.
```

THE MENTAL MODEL

Think of the checkpointer as a filing cabinet and each `thread_id` as a labelled folder. Every `invoke` opens the right folder, adds the new pages, and files it back. To resume a conversation tomorrow, just pass the same id again.

5.7 — Keeping context from overflowing

Conversations grow. Eventually the message list can exceed the model's **context window** (the max tokens it can read at once) — causing errors, higher cost, and worse answers. Three standard fixes, all built into LangChain's middleware:

Strategy	What it does	Trade-off
Trim	Keep only the last N messages (or last N tokens) before each model call.	Simple; loses old details.
Delete	Permanently remove specific messages from the state.	Precise control; manual.
Summarize		

	Replace old turns with an LLM-generated summary, keeping the gist.	Best recall; costs an extra call.
--	--	-----------------------------------

THE EASY BUTTON: SUMMARIZATIONMIDDLEWARE

LangChain ships a `SummarizationMiddleware` that automatically condenses old history once it crosses a token threshold — you attach it and forget about it. We'll wire middleware in properly alongside agents in Phase 9. For this phase, just know the lever exists: when a long chat starts misbehaving, reach for trim or summarize rather than letting history grow unbounded.

5.8 — Common errors & troubleshooting

Symptom	Likely cause & fix
Bot doesn't remember anything across calls	You forgot the <code>config</code> with a <code>thread_id</code> , or used a <i>different</i> id each call. Pass the same <code>{"configurable": {"thread_id": ...}}</code> every turn.
You compiled without a checkpointer and have no memory	<code>builder.compile()</code> with no <code>checkerpointer=</code> gives a stateless graph. Add <code>checkerpointer=InMemorySaver()</code> .
Memory vanishes when the program restarts	That's expected with <code>InMemorySaver</code> (RAM only). Use a <code>SqliteSaver</code> / <code>PostgresSaver</code> to persist to disk/DB.
<code>ModuleNotFoundError: langgraph</code>	<code>langgraph</code> isn't installed. <code>pip install -U langgraph</code> (it was in the Phase 0 list).
Copied a tutorial using <code>ConversationChain</code> / <code>RunnableWithMessageHistory</code> and got deprecation warnings/errors	Those are legacy (§5.2). Re-implement with the checkpointer pattern in §5.5.
Long chats slow down or error out	History outgrew the context window. Trim or summarize (§5.7).

5.9 — Mini practice exercise

YOUR TURN (DO THIS BEFORE PHASE 6)

Task: Build an interactive command-line chatbot with memory. Use the LangGraph checkpointer pattern (§5.5). Wrap it in a `while True` loop that reads input with Python's `input()`, sends it under a fixed `thread_id`, prints the reply, and exits when the user types `quit`. Give it a personality with a `SystemMessage`. Then prove threads work: start the script, tell it a fact about yourself, quit, restart — and (because `InMemorySaver` is RAM-only) watch it forget. Bonus: swap in a `SqliteSaver` so it remembers across restarts.

Why: this is the first genuinely *interactive* app you'll build, and it cements the difference between in-memory and persistent storage — the exact distinction that matters when you deploy in Phase 11.

Hints (not the full answer):

- Keep the graph from §5.5 exactly; only the calling code becomes a loop.
- To include a persona on every turn without re-sending it, put a `SystemMessage` as the very first message of the thread (send it once on the first turn), or add it inside the node before `invoke`.
- For the bonus: `pip install langgraph-checkpoint-sqlite`, then `from langgraph.checkpoint.sqlite import SqliteSaver` and follow its `from_conn_string` usage — the rest of your code is unchanged.
- Use `r["messages"][-1].content` to print just the latest reply.

Terms introduced this phase

Term	One-line meaning
Stateless	The model remembers nothing between calls; it sees only the current request.
Message history	The accumulated list of turns you resend so the model has context.
Graph (LangGraph)	A small workflow of nodes; a chatbot needs just one (call the model).
State / MessagesState	Data the graph carries; <code>MessagesState</code> holds an auto-appending <code>messages</code> list.
Node / edge / START	A step / a connection between steps / the graph's entry point.
Checkpoint	Saves graph state between calls; <code>InMemorySaver</code> (dev), <code>SqliteSaver</code> / <code>PostgresSaver</code> (prod).
Thread / thread_id	One independent conversation; the key memory is stored under.
Context window	The max tokens a model can read in one call; history must fit inside it.
Trim / summarize	Strategies to shrink long history so it stays within the context window.
Legacy memory classes	<code>ConversationChain</code> , <code>*Memory</code> , <code>RunnableWithMessageHistory</code> — deprecated; use a checkpoint.

PHASE 5 COMPLETE

You understand that models are stateless and memory is just resent history — and you can implement it both by hand and the modern way, with a `LangGraph` checkpoint that gives you persistence and per-thread separation for free. You also know the deprecated classes to steer clear of. **Next — Phase 6: Loading & Splitting Documents**, where we start feeding the model your *own* data: loading PDFs, text, and web pages, then splitting them into clean chunks — the first half of building a RAG system.